Web Applications

TypeScript

Suhel Hammoud

TypeScript for JavaScript Programmers

TypeScript extends JavaScript by adding a powerful type system. It catches errors early, making your code more reliable.

- Static typing helps detect bugs during development
- Improves IDE support (autocomplete, refactoring)
- Scales better for large applications

The Basics

- TypeScript is a superset of JavaScript
- You can gradually adopt it
- Compiles to plain JavaScript

```
let message: string = "Hello";
let count: number = 10;
```

- Types are erased at runtime

Everyday Types

Common types used daily:

- string, number, boolean
- Arrays: string[]
- Objects: {}

```
let age: number = 30;
let isAdmin: boolean = true;
let names: string[] = ["Alice", "Bob"];
```

- These are the building blocks of all programs

Types by Inference

TypeScript infers types automatically.

```
let helloWorld = "Hello World";
```

- Reduces verbosity
- Keeps code readable
- Still provides full type safety

Defining Types

When inference is not enough, define types explicitly.

- Improves readability
- Documents intent

```
const user = {
  name: "Hayes",
  id: 0,
};
```

```
interface User {
  name: string;
  id: number;
}

const user: User = {
  name: "Hayes",
  id: 0,
};
```

TypeScript Types Overview

TypeScript provides a rich type system for safer and more expressive code.

- Types describe the shape of data
- Enable better tooling and error checking
- Useful for large-scale applications

Type Aliases

Type aliases give names to types:

```
type UserID = string;
type Point = { x: number; y: number };
```

- Improves readability
- Reusable across code

▓▓ Type vs Interface

Both describe shapes, but differ:

- `interface` → best for object shapes
- `type` → more flexible

```
interface User {
  name: string;
}

type Admin = User & { role: string };
```

- Types support unions, intersections, and more

Object Literal Types

Define object structures inline:

```
type Product = {
  name: string;
  price: number;
  inStock?: boolean;
};
```

- Optional fields with ?
- Strong structure enforcement

Function Types

Describe callable structures:

```
type Callback = (value: string) => void;
```

- Can define parameters and return types

Union Types

A value can be one of many types:

```
type Size = "small" | "medium" | "large";
```

- Useful for fixed sets of values

Intersection Types

Combine multiple types:

```
type A = { x: number };
type B = { y: number };

type C = A & B; // { x: number, y: number }
```

- Merges properties

Tuple Types

Fixed-length arrays with known types:

```
type Data = [number, string];
```

- Each position has a specific type

Type from Value

Extract types from existing values:

```
const data = { name: "Alice" };

type Data = typeof data;
```

- Keeps types in sync with values

Indexed Access Types

Access part of a type:

```
type Response = { data: string };

type Data = Response["data"];
```

- Extracts nested types

▓▓ Conditional Types

Types with logic:

```
type IsString<T> = T extends string ? true : false;
```

- Works like `if` in type system

Template Literal Types

Build types using strings:

```
type Lang = "en" | "pt";
type Section = "header" | "footer";

type IDs = `${Lang}_${Section}_id`;
```

- Combines string unions

▓▓ Mapped Types

Transform existing types:

```
type Subscriber<T> = {
  [K in keyof T]: (value: T[K]) => void;
};
```

- Iterates over properties
- Creates new structures

Utility Types

- Create new types from existing ones
- Built-in helpers:
  - `Partial<T>`
  - `Readonly<T>`
  - `Pick<T, K>`
  - `ReturnType<T>`
- Simplify common patterns
- Avoid duplication
- Improve maintainability

Partial Type

```
type User = { name: string; age: number;};
type PartialUser = Partial<User>;
```

Constructs a type with all properties of 'User' to optional

Readonly Type

```
type Point = { x: number; y: number };
type ReadonlyPoint = Readonly<Point>;
```

Constructs a type with all properties of 'Point' set to readonly

Keyof Type Operator

```
type User = { name: string; id: number };
type Keys = keyof User;
```

- Produces union of keys

Typeof Type Operator

```
const user = { name: "Alice" };
type UserType = typeof user;
```

- Extracts type from value

▓▓ Pick from type

Syntax: `Pick<OriginalType, 'key1' | 'key2'>`

```
interface User {
  id: number;
  name: string;
  email: string;
  password: string;
}

// Pick only public fields, exclude password one
type PublicUser = Pick<User, 'id' | 'name' | 'email'>;
```

Return Type Extraction

Get function return types:

```
function createUser() {
  return { name: "Alice" };
}

type User = ReturnType<typeof createUser>;
```

TypeScript Classes Overview

TypeScript builds on JavaScript classes with additional type features.

- Adds type safety to class members
- Includes compile-time checks
- Keeps JavaScript runtime behavior

Defining a Class

Basic class syntax:

```
class User {
  name: string;
  id: number;

  constructor(id: number, name: string) {
    this.id = id;
    this.name = name;
  }
}
```

- Constructor initializes fields
- Types are enforced during development

Creating Instances

Create objects using `new`:

```
const user = new User(1, "Alice");
```

- `new` calls the constructor
- Returns an instance of the class

▓▓ Access Modifiers

Control visibility of properties:

- public (default)
- private
- protected

```
class Account {
  private balance: number;

  constructor(balance: number) {
    this.balance = balance;
  }
}
```

- private is only checked at compile time

▓▓ private vs #private

Two types of privacy:

```
class Example {
  private x = 1;   // Type-only
  #y = 2;          // Runtime private
}
```

- `private` → TypeScript only
- `#private` → Enforced in JavaScript runtime

Methods and this

The value of this depends on how a function is called.

```
class Counter {
  count = 0;

  increment() {
    this.count++;
  }
}
```

- Use arrow functions or bind to fix this

```
  increment = () => this.count++;
```

Getters and Setters

Control access to properties:

```
class User {
  private _name: string = "";

  get name() {
    return this._name;
  }

  set name(value: string) {
    this._name = value;
  }
}
```

- Encapsulates logic
- Provides controlled access

Static Members

Belong to the class, not instances:

```
class Config {
  static version = "1.0";

  static getVersion() {
    return Config.version;
  }
}
```

- Access using class name
- Shared across all instances

▓▓ Parameter Properties

Shortcut for defining and initializing fields:

```
class Point {
  constructor(public x: number, public y: number) {}
}
```

- Automatically creates and assigns properties

Inheritance

Extend classes using `extends`:

```
class Animal {
  speak() {
    console.log("Some sound");
  }
}

class Dog extends Animal {
  speak() {
    console.log("Bark");
  }
}
```

- Enables code reuse
- Supports method overriding

Implements

Ensure a class follows a structure:

```
interface Serializable {
  serialize(): string;
}

class User implements Serializable {
  serialize() {
    return "user";
  }
}
```

- Enforces contracts

Abstract Classes

Cannot be instantiated directly:

```
abstract class Animal {
  abstract getName(): string;

  printName() {
    console.log("Hello " + this.getName());
  }
}
```

- Used as base classes
- Can include abstract methods

Generics in Classes

Reusable type-safe classes:

```
class Box<T> {
  contents: T;

  constructor(value: T) {
    this.contents = value;
  }
}
```

- Works with different types
- Improves flexibility

Structural Type System

- TypeScript checks structure, not name

```ts
interface Point {
  x: number;
  y: number;
}

function logPoint(p: Point) {
  console.log(`${p.x}, ${p.y}`);
}

const point = { x: 12, y: 26 };
logPoint(point);
```

Shape Matching

```
const point3 = { x: 12, y: 26, z: 89 };
logPoint(point3);

const color = { hex: "#187ABF" };
logPoint(color); // Error
```

Classes and Structural Typing

```
class VirtualPoint {
  x:number; y: number;

  constructor(x: number, y: number) {
    this.x = x;
    this.y = y;
  }
}

const newVPoint = new VirtualPoint(13, 56);
logPoint(newVPoint);
```

Decorators

Add metadata to classes and members:

```
@sealed
class User {
  name: string;
}
```

- Used for frameworks and advanced patterns.
- Common Use Cases: Logging, Validation, Caching, Serialization, etc.
- Require "experimentalDecorators": true in tsconfig.json.

References

- https://www.typescriptlang.org/docs/handbook